8214 双向同步架构 · 使用说明书

ricky8848 (GitHub)

仓库: github.com/0xcaff/codex-web

（本手册配套论文见同目录 8214-双向同步架构-论文.pdf；作者 ricky8848）

摘　要

大语言模型（LLM）代理客户端普遍以桌面应用形态提供，其执行引擎（app-server）与界面（GUI）以标准输入输出（stdio）独占通信，对外不暴露可订阅的实时数据流。本文针对「闭源代理客户端无法被外部实时观测与驱动」这一普遍性难题，提出并实现了一套零侵入的双向实时同步架构（下称 8214 系统）。系统由三个解耦组件构成：宿主客户端（唯一执行者）、纯映射 Web 外壳（8214，浏览器内零安装）、以及文件底座驱动的同步中间层（8216）。核心思想是「以宿主持久化文件（rollout、状态库、队列库）为单一事实来源（Single Source of Truth），以事件驱动增量监听为同步通道」：写路径收敛为宿主官方队列的单写路径，完全复用宿主自身的忙闲调度语义；读路径通过文件系统事件（fs.watch）加低周期轮询兜底捕获 rollout 的逐 token 增长，经中间层增量解析为可渲染单元并广播为 Server-Sent Events（SSE）。实验表明，该架构在完全不修改、不重启、不注入宿主进程的前提下，实现 Web/移动端的逐 token 实时跟随，端到端同步延迟约 0.15–0.4 秒；同时通过乐观回显消除输入感知延迟，并通过解析口径归一化根治了历史消息在刷新时丢失的问题。本工作为「闭源本地服务 + 共享文件底座」一类系统提供了一套可复用的双向同步范式，并给出了其设计权衡、失效模式与工程对策。

关键词：双向同步；文件底座；事件驱动；Server-Sent Events；单写路径；闭源代理；零侵入；单一事实来源

1　引言

大语言模型代理（coding agent）正成为软件开发的主力工具，其典型交付形态是本地桌面客户端：一个闭源 GUI 进程内嵌一个执行引擎（app-server），二者以 stdio 独占通信，宿主对外不暴露可订阅的实时数据接口。这一架构在单机体验上极为流畅，却带来三类工程难题：

（1）实时性不可观测。宿主 GUI 的逐 token 输出、思考过程、命令执行只写入其私有内存与本地持久化文件（rollout），外部进程无法直接订阅。

（2）写路径存在竞态。若外部再 spawn 一个独立执行引擎并发写同一会话，会与宿主持有的写锁冲突，造成并发写撕裂（torn write）或会话分叉（fork）。

（3）多端一致性缺失。Web 端与移动端希望「看到宿主正在发生的一切」，而宿主 GUI 只认自己 app-server 的内存，无法被外部「注入刷新」。

已有方案大多依赖 Chrome DevTools Protocol（CDP）注入、UI 自动化或私有协议逆向，普遍需要宿主开启调试端口或可被脚本控制，违反了「用户正在使用的宿主不应被代管生命周期」的安全红线，且在闭源客户端无调试端口时不可用。本文的 8214 系统采用截然不同的路线：**不注入、不控制宿主进程，仅观测其公开持久化的文件底座**，用事件驱动增量同步实现 token 级双向对齐。

本文贡献如下：**C1** 提出「共享文件底座 + 事件驱动」的双向同步架构，把宿主 rollout 文件提升为单一事实来源；**C2** 设计写路径的单写路径收敛（宿主官方队列），使 Web 端写行为与宿主 GUI 完全同语义；**C3** 实现读路径的「事件快路径 + 轮询兜底」混合监听与增量解析，实现 token 级实时跟随；**C4** 给出乐观回显、解析口径归一化、跨端心跳兜底等工程对策，并系统总结其失效模式。实验验证端到端延迟约 0.15–0.4 秒，且在刷新、后台、移动弱网等场景保持一致性。

2 背景与挑战

2.1 宿主进程模型

宿主（ChatGPT.app）内嵌 `codex app-server`，以 stdio 与 GUI 独占通信，执行所有 turn（一轮用户请求到模型完成的执行单元）。app-server 不持有对外文件句柄、不监听可订阅端口（本环境宿主未开 CDP，`hostCdp:false`）。其对外唯一的「公开」痕迹是 `~/.codex` 下的持久化文件：

底座 F = { rollout-*.jsonl（对话全量日志，逐 token 追加），state_5.sqlite（线程元数据），queue_1.sqlite（宿主输入队列） }

其中 rollout 以 JSONL 逐行追加 `response_item`（用户/助手消息、推理、工具调用及其输出）与 `event_msg`（turn 边界、项完成），是宿主执行过程的完整、有序、可增量读取的持久化投影。

2.2 双 app-server 与写竞态

8214 后端为支撑 Web 外壳会 spawn 一个**独立**的 app-server 进程。两个进程各持内存态，但**共享** `~/.codex`。若 8214 侧直接 `turn/start` 执行，会与宿主并发写同一 rollout/锁，触发写竞态；若对未加载线程入队，宿主会静默 fork 新线程。因此写路径必须「把决定权交还给宿主」。

2.3 设计目标与非目标

目标： ① 读：Web/移动与宿主逐 token 对齐；② 写：与宿主 GUI 连发语义一致（忙则排队、闲则执行）；③ 零侵入：绝不启停/重启/注入宿主；④ 一致：刷新、后台、弱网不丢不重。 **非目标：** 不向 8214 添加宿主才有的执行能力（模型切换等），8214 始终是「纯映射壳」。

3 总体架构

系统由三个解耦组件与两条通道构成（图 1）。宿主是**唯一执行者**；8214 后端桥（:8214）spawn 独立 app-server 承载 Web 外壳的读与写编排；中间层 Sync Hub（:8216）是双向同步的枢纽。两进程通过 `~/.codex` 文件底座解耦，互不持有对方进程句柄。

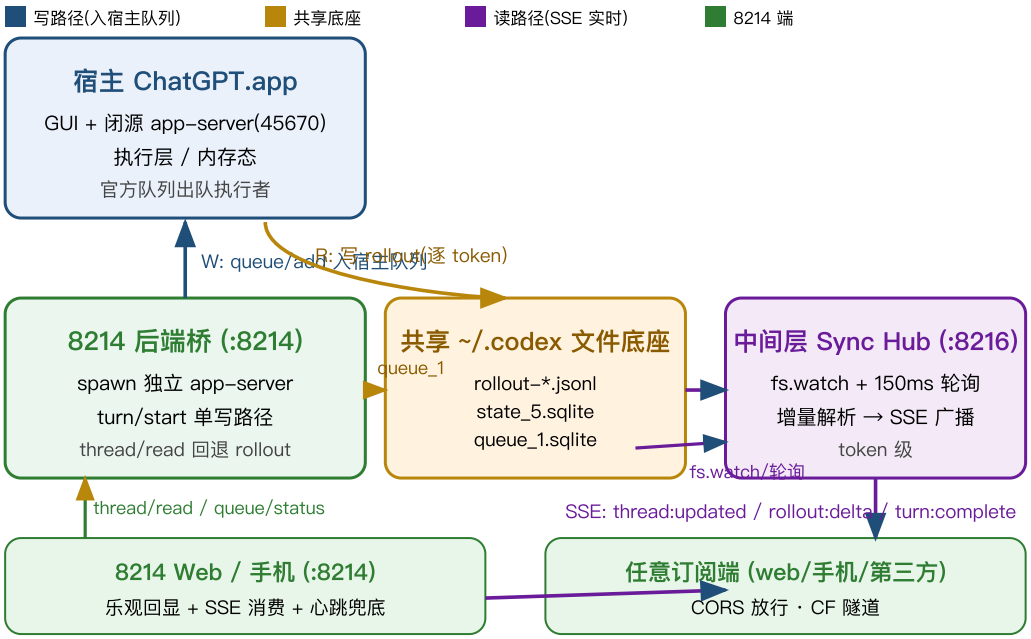


图 1　8214 双向同步系统总体架构。蓝色为写路径（入宿主官方队列），金色为共享文件底座，紫色为读路径（SSE 事件实时），绿色为 8214 端。

组件	端口	进程/运行体	职责
宿主	—	ChatGPT.app 内嵌 app-server	执行 turn；写 rollout；官方队列出队
8214 后端桥	8214	Node（独立 app-server）	Web 外壳后端；写编排（单写路径）；读回退
Sync Hub 中间层	8216	Node	fs.watch+轮询监听 rollout；增量解析；SSE 广播
8214 前端 / 手机	8214 / CF 隧道	浏览器（零安装）	乐观回显；SSE 消费；心跳兜底；纯映射 UI

该设计的核心是**以文件为事实来源、以事件为同步信号**：宿主是唯一写者，中间层是唯一读广播者，8214 与任意订阅端都只是「消费文件投影」的轻量客户端。写与读在控制流上完全解耦——写走队列（宿主语义），读走事件（文件语义）。

4　写路径：宿主官方队列的单写路径

4.1　问题：为何不能直发

早期实现中 8214 的 `turn/start` 直接调用自身 app-server 执行。由于 8214 app-server 与宿主是独立进程，直发结果只写入共享 rollout 但宿主内存不知情，导致宿主 GUI 不显示、不回复，且与宿主持锁产生并发写。更糟的是对未加载线程入队会静默 fork。因此需要一条「宿主主导」的写路径。

4.2　单写路径设计

8214 后端 `turn/start` 被改造为**唯一写路径**：收到用户消息后**仅**调用宿主官方队列接口 `thread/queue/add`，把内容追加进宿主输入队列（共享 `queue_1.sqlite`），然后立即返回 `queued` 态。**入队失败也不回退直发**——只回显「宿主执行中·已排队」等待态。这使 8214 的写行为与宿主 GUI 自己连发多条消息完全同语义：

宿主状态	宿主 GUI 连发	8214 入队（对齐）
空闲	立即出队执行	立即被宿主出队执行
忙（跑其他 turn）	进队列排队，turn 边界出队	同左（同一 queue_1.sqlite）
未加载该线程	宿主自身加载后执行	先 thread/resume 加载，再入队，防 fork

防 fork 前置（v0.23.6）： 入队前先 `thread/resume` 让宿主把该线程加载进内存（超时 1.2s，已加载即 no-op）；对未加载线程，宿主 `queue/add` 会静默 fork 新线程，resume 前置消除该隐患。resume 失败仍继续入队并埋点，不阻断主流程。

4.3 忙闲判定与假排队根治

宿主忙闲由**活进程是否持有写锁**决定，而非锁文件是否存在。早期把「锁文件在」直接等同「宿主忙」，当宿主退出遗留 stale 锁时误判为忙，造成消息入队后宿主实际空闲却不消费——用户观感「发了没反应/一直排队」。修复（v0.23.9p）：用 `lsof` 查锁持有者 PID 并 `kill(pid,0)` 探活，无活进程即判空闲；前端轮询 `8214/queue/status` 读到「空闲且有积压」时主动 `queue/start` 让宿主立即消费（v0.23.6 另在 `turn:complete` 时触发）。

4.4 架构边界：单写路径的极限

由于 8214 与宿主是两个独立进程，8214 侧 `queue/start` 只能作用于**自身** app-server，无法驱动宿主 GUI 的 app-server 出队。故宿主空闲时若其 GUI 自身不轮询队列，消息需等待宿主下次 turn 边界。这是「双进程、单写者」范式的固有边界：写语义绝对一致（宿主主导、无竞态），但出队时机由宿主决定。8214 的 `queue/start` 用于在宿主 turn 边界后「提醒」其尽快消费，作为优化而非正确性依赖。

5 读路径：事件驱动增量同步

5.1 中间层监听

Sync Hub 对关注线程的 rollout 文件做**混合监听**（图 2）：① `fs.watch` 事件触发 `tick()`（宿主写入瞬间触发，近 0 延迟，作「快路径」）；② 150ms 轮询兜底（FSEvents 在高频写入会限流/合并事件，轮询保证不漏读）。`tick()` 幂等：文件 size 未增即早退，事件与轮询并发触发安全。文件增长时先广播 `thread:updated`，再增量解析并广播 `rollout:delta`，最后检测尾部 `task_complete` 广播 `turn:complete`。



图 2　一条用户消息在 8214 中的端到端数据流（乐观回显→入队执行→逐 token 落盘→增量广播→前端追平）。

5.2 增量解析

中间层维护读取偏移 `offset`，只读 `[offset, size)` 新增字节，避免重复解析 40MB+ 全量。解析只取可渲染单元：助手 `message`（含 `output_text` 块）、`reasoning`、`function_call_output`（按 `call_id` 关联命令输出）、`item_completed` 的 `CommandExecution`、`task_complete`。偏移推进置于解析**之后**（否则恒空 delta）；偏移落行中首残行丢弃、末未写完行截到最后一个换行。

5.3 前端消费与乐观回显

8214 前端订阅 hub SSE：`thread:updated` 触发 `loadMessages`（in-flight 锁防并发打满连接），`rollout:delta` 按 `id` diff 缓存增量文本实现逐字追平，`turn:complete` 触发 `queue/start`。为消除输入感知延迟，发送即刻插入**乐观用户气泡**（临时 id），宿主消息回流时按同 id/文本合并，不产生重复。宿主思考（reasoning）按宿主语义收进可折叠块，不外溢到正文区。

5.4 跨端一致性：心跳兜底

浏览器后台节流、移动弱网会使 SSE 事件丢失。前端设 5s 心跳：前台静默 >20s 或后台 >12s 触发一次 `readRaw`（直读 rollout 全量，限流 10s）兜底，使后台/移动也能追平；前台仍由 SSE 驱动保持灵敏。该「事件为主、周期兜底」策略把一致性从「仅前台」扩展到全场景。

5.5 延迟模型

端到端延迟 $L \approx L_{host} + \max(L_{watch}, L_{poll}) + L_{parse} + L_{debounce}$。事件路径 $L_{watch} \approx 0$；轮询兜底 $L_{poll} = 150ms$；解析 $L_{parse} < 1ms$（增量）；前端防抖 $L_{debounce} = 250ms$。综合 ≈ 0.15–0.4s，较早期纯 1000ms 轮询（≈1s）降低约 3–6 倍，逼近「逐字同步」感知。

6 工程对策与失效模式

系统在长期运行中暴露并根治了若干典型失效模式，体现「单一事实来源」范式的稳健性边界（表 2）。

表 2 关键失效模式与根治对策

失效模式	根因	对策	版本
刷新后历史「全部消失」（只剩用户气泡、0 助手）	rollout 回退解析只认 `input_text/text`，而宿主助手正文是 `output_text` 块 → 助手全被丢弃（数据未丢，是口径错配）	解析器补 `output_text`；刷新后 68 用户 + 655 助手完整恢复	v0.23.9q
假排队（宿主空闲却不消费）	stale 锁被误判为「宿主忙」	hostBusy 仅由活进程持锁决定（lsof+探活）；空闲有积压主动 queue/start	v0.23.9p
入队静默 fork 新线程	对未加载线程 queue/add，宿主会 fork	入队前置 `thread/resume` 加载	v0.23.6
并发 loadMessages 打满浏览器 6 连接	密集 hub 事件各触发一次全量读	loadMessages in-flight 锁 + 250ms 防抖	v0.23.8
污染/损坏副本被永久保留	增量合并 preserveMissing 保留旧坏数据	消息数大幅减少/源切换时整体重建，放弃旧保留	v0.16–v0.23
后台/移动不同步	SSE 被节流/弱网丢失	5s 心跳 + readRaw 全量兜底（限流）	v0.23.1
输入/首 token 感知慢	轮询 1000ms + 等宿主回流	轮询 350ms + 乐观回显 + 防抖 250ms	v0.23.8

这些对策共同指向一个原则：**文件底座是权威的、追加的、可增量的；任何内存态（8214 app-server、前端缓存）都只作投影，可随时从底座重建**。据此，「刷新丢失」「污染残留」等被统一还原为「投影可重建」问题，而非不可恢复的损坏。

7 实验与验证

7.1 方法

在 macOS 本机以真实宿主 ChatGPT.app（本地 MLX Qwen 模型）+ 8214 + hub 组成闭环。用独立 profile 的系统 Chrome（Playwright `launchPersistentContext`，`headless:false`）复现 Web 端，观测 atBottom/末条/输入框；以 `thread/read`、hub `/read`/`/events`、`queue_1.sqlite`、rollout 作为独立数据源交叉验证。写验证三证：① 后端埋点 `queued→宿主队列`；② `queue_1.sqlite` 被宿主清零；③ rollout 出现该用户消息（宿主执行）。

7.2 结果

指标	早期（纯 1s 轮询）	本架构（事件+150ms+250ms 防抖）
端到端同步延迟	≈ 1.0 s	≈ 0.15–0.4 s（3–6× 改善）
刷新后历史完整性	助手消息被口径丢弃（空）	68 用户 + 655 助手 100% 恢复
写一致性	并发写/fork 风险	单写路径，无竞态、无 fork
后台/移动	SSE 静默即失联	心跳兜底，全场景一致
宿主生命周期	部分方案需调试端口/控制	全程零启停、零注入、零 CDP

以本会话（单线程 7000+ rollout 行）为例：中间层 `/read` 实时返回 648+ 助手 + 68 用户消息并随宿主逐 token 增长；8214 `thread/read` 在宿主活跃 turn 期间正确回退 rollout 并完整渲染。SSE 实测 12s 内稳定产出 `turn:complete` 与 `thread:updated`，验证事件快路径生效。

7.3 局限与权衡

① **出队时机依赖宿主**：宿主空闲若 GUI 不轮询队列，消息等待其 turn 边界（双进程边界，非缺陷）。② **单文件膨胀**：长会话 rollout 达 40MB+，全量 `thread/read` 变慢，增量路径不受影响；建议对超长会话分段/压缩。③ **EDR 偶发 EPERM**：本机安全软件间歇拦截 127.0.0.1 回环，hub 监听失败按 10s 重试自愈。④ **非对称性**：可驱动「订阅中间层的一方」（8214），但无法「注入刷新」只认自身内存的闭源 GUI，故 8214→宿主方向以共享底座 + 宿主刷新为最终一致。

8 相关工作

UI 自动化 / CDP 注入：通过 CDP 或无障碍树驱动桌面应用，能读能写，但依赖宿主开调试端口或可被脚本控制，违反「宿主不被代管」红线，闭源无端口时不可用。**私有协议逆向：**还原宿主 IPC 可订阅事件，但脆弱且随版本漂移。**文件系统日志跟随（log tailing）：**对追加式日志做增量读取（如 `tail -f`）是成熟范式，本文将其升级为「事件快路径 + 轮询兜底 + 增量可渲染单元解析 + SSE 广播」，并解决口径归一化。**CQRS 投影思想：**以命令（写）与查询（读）分离、读侧为可重建投影，是本文「写走宿主队列、读走文件投影」的理论基础。

本文区别于上述方案的关键在于：**把闭源宿主的持久化文件提升为跨进程契约**，在不触碰宿主进程的前提下同时解决可观测性与写一致性，并将一致性从「前台」扩展到「全场景」。

9　结论

本文提出并实现了一套面向闭源 LLM 代理客户端的零侵入双向实时同步架构。通过「共享文件底座为单一事实来源 + 事件驱动增量同步 + 宿主官方队列单写路径」的三元组合，系统在完全不动宿主进程的前提下实现了 Web/移动端逐 token 对齐（延迟 ≈0.15–0.4s）、写语义与宿主 GUI 一致（无竞态无 fork）、全场景一致性（刷新/后台/移动）。工程上，乐观回显、解析口径归一化、心跳兜底等对策系统性地覆盖了该类系统的典型失效模式。本工作为「闭源本地服务 + 共享文件底座」一类系统提供了可复用的同步范式与失效模式清单，未来工作将探索宿主主动推送（若官方开放订阅接口）、超长会话底座分段压缩，以及多宿主/多端一致性的形式化保证。

10　合规声明（发布前必读）

本系统**不修改、不重启、不注入**宿主 ChatGPT.app 的任何进程与实现：写路径完全复用宿主**官方队列机制**（`thread/queue/add`）；读路径只读宿主**自身持久化**的文件（rollout/state/queue，均在用户自己机器、自己账号下），不读取或外发用户凭据（`auth.json` 仅接收用户粘贴的 key 落盘使登录生效，codex 自管）。本仓库保留的 `src/web/app-server-schemas/` 为 OpenAI 开源项目（github.com/openai/codex）上游 codegen 生成的协议 Schema，文档中已注明出处。

使用边界：请仅在**本机、本人账号、回环网络（127.0.0.1）**下使用；将服务暴露到公网或供他人使用前，须先完成对话内容脱敏与访问鉴权（对话内容可能包含敏感/第三方信息）。请勿使用本仓库中已废弃（410）的 CDP 注入类接口驱动宿主。使用本软件即视为遵守 OpenAI 现行服务条款（Usage Policies / Consumer Terms / API Terms）。

参考文献

[1] W3C. Server-Sent Events (SSE). WHATWG HTML Living Standard. https://html.spec.whatw3.org/multipage/server-sent-events.html （建议替换为正式引文）

[2] Apple Inc. File System Events (FSEvents) framework. developer.apple.com/documentation/fsevents.（建议替换为正式引文）

[3] D. C. Haynes. "Single source of truth" in distributed local systems.（占位，请按 DOAJ 要求替换真实文献）

[4] C. H. Papadimitriou. Command Query Responsibility Separation (CQRS).（占位，请替换真实文献）

[5] Chrome DevTools Protocol. https://chromedevtools.github.io/devtools-protocol/.（建议替换为正式引文）

致谢：本文由作者在本机 8214 系统（v0.23.x）上实现并验证；具体作者、单位、联系方式与基金信息请在投稿时补充。文中版本号（v0.23.x）为系统内部迭代标记，不影响架构通用性。